

Typed eDSL — realigning with Tidal's two-layer model, but *typed*

Supersedes `neutral-accent-plan.md` (the B1/B2 framing). Accents stop being a feature to bolt on and become a single field of a typed payload.

The decision

We will **not** adopt Tidal's stringly-typed `ValueMap` (`Map String Value`) — a stringly core is a non-starter for the composition/generative ambitions, where we want total functions, exhaustive matches, and compile-time-checked control names.

But we *will* reverse the accidental design that "fell out" of the parsers-for-a-PoC phase, and realign with Tidal's **two-layer model** (parse → controls → merge with `#`) — implemented over a **typed control record** instead of a string-keyed map. Net: Tidal's composability, *more* principled than OG Tidal, not less.

What's wrong today (diagnosis)

purel-tilde has two incoherent notions of "what a pattern event is":

- **Family A — parsers** (`String → MiniNotation a`, same grammar, different value type):
`mini/drum/miniTyped → MiniNotation String; degree/pitch → MiniNotation PitchedNote12`. `drum` is literally `= miniTyped`.
- **Family B — control-lifts** (`Pattern x → ControlPattern`, i.e. `Pattern ValueMap`):
`s/n/note/gain` in `Tidal.Controls` — **orphaned**, used by nothing on the part path.

The part path consumes **Family A directly** (a typed value pattern) and lets the **binding** (instrument/kit: channel, default velocity) stand in for what Tidal expresses as controls. So Family B — the entire `# control` world, accents included — has nowhere to attach. The two families carry *different payloads* (typed value vs stringly map) and only one is wired. That's the incoherence.

The surface redundancy ("`drum`" vs "`s`") is **fine** and we keep it. The fix is to give both families **one typed payload** to converge on.

The target — one typed `Sound` payload

```
-- The typed event payload. Every control is an optional, typed field; the
-- vocabulary is bounded and known. Adding a control = adding a field
-- (checked).
-- NO string keys, NO escape hatch.
type Sound =
  { source :: Maybe Token      -- "bd", "arpy", a kit hit / sample bank -
    MEANING from the binding
  , index  :: Maybe Int       -- sample slot within a bank (Dirt `n`,
    Rample slot, Digitakt sample)
  , pitch  :: Maybe Pitch     -- melodic pitch; Degree stays late-bound
    (live scale)
```

```

    , gain    :: Maybe Number    -- 0..1 neutral loudness → MIDI vel / Dirt
    amp / modular accent
    , pan     :: Maybe Number
    , speed  :: Maybe Number    -- sample playback rate / transpose-by-rate
    , begin   :: Maybe Number    -- sample start 0..1
    , end     :: Maybe Number    -- sample end 0..1
    , cutoff  :: Maybe Number
    , shape   :: Maybe Number
    -- ... the common, bounded set. That's the whole vocabulary.
  }

data Pitch
  = Degree Int      -- resolved against the active scale AT EMIT — keeps
  the live-scale re-render
  | Note Number     -- absolute semitone
  | Chromatic Int   -- raw MIDI note

type SoundPattern = Pattern Sound

```

Token is a newtype over `String` (a parsed mini-notation token), not a bare `String` — so "source" and "a number" can't be confused.

How the vocabulary collapses into one coherent layer

Every verb yields the **same type**, **SoundPattern**:

- **s, sound, drum** — set **source**. Same operation; **the binding decides what a token means** (on **vDrums kit ...** → kit note; a sampler binding → slot; a Dirt binding → sample). This is "two ways to say the same thing," now genuinely the same thing with the same type. (**drum = s <<< mini, sugar.**)
- **degree, note, n, pitch** — set **pitch** (typed). **degree "0 2 4"** → **pitch = Degree ...**; the active-scale render is just "resolve **pitch** at emit."
- **gain, pan, speed, begin, end, ...** — typed control combinators, each setting its field.
- **#** — a **typed, right-biased record merge** (**mergeRight**): structure from the left, fields from the right win where set. No string keys. **|+|**, **|*|** etc. can follow as typed numeric merges later.

So **drum "bd sn" # gain "1 0.6"** is a typed merge of two **SoundPatterns** — and **accents are simply the gain field**, no special mechanism.

Mini-notation literals are fine — that is *not* the stringliness we rejected

gain "1 0.6" reads **"1 0.6"** as **mini-notation** (a **Pattern Number**), which is the value DSL we keep. The thing we rejected is stringly *control keys* (**Map.lookup "gain"**), not mini-notation string literals. Decide separately whether to add **IsString (Pattern a)** so **"1 0.6"** parses directly (ergonomic, Tidal-like) or require **gain (nums "1 0.6")** (explicit). Orthogonal to the type model.

Bindings render the typed **Sound** to the substrate

A voice = a binding (where it goes) + a **SoundPattern** (what it plays). The binding reads **typed fields** (no lookups) and renders per target — this is the multi-output substrate, now typed:

Sound field	MIDI synth	MIDI drum kit	SuperDirt (OSC)	ES-9 gate/CV	Sampler (Rample/Digitakt)
pitch	note (Degree→scale)	—	note/n	V/oct CV	(slot transpose)
source	—	hit → kit note	s	—	bank/track
index	—	—	n	—	slot/sample
gain	velocity	velocity	gain/amp	accent gate / vel- CV	velocity / level
speed/begin/end	—	—	Dirt params	—	rate / start CV

The sample-trigger abstraction (consider now, build optionally)

The **source/index/gain/speed/begin/end** subset is a generic sample-trigger vocabulary shared by SuperDirt, Squarp Rample, and Digitakt: "play sample X from bank/track B at level G, rate R, from start S." A single **SoundPattern** could bind to any of them; the **binding** does the target-specific mapping (Dirt OSC keys / Rample MIDI+CV+gate / Digitakt MIDI note+CC). This is why **index/begin/end** earn fields now even though MIDI synths ignore them.

Honest hard edge: **slot/sample addressing differs per device** (Dirt **n** index vs Rample channel+slot vs Digitakt track+sample). The common fields cover level / rate / start; addressing is where the abstraction may prove too restrictive. Keep it as a binding concern, design the fields to allow it, don't over-commit. Fine if we find it's not worth fully unifying.

What is preserved (not lost)

- **Live active-scale re-render** — **Degree** stays a typed field resolved at emit against **Window.activeScale**. Unchanged behaviour, now a field of **Sound**.
- **Per-voice supervision / hot-reload / per-cell compile** — BEAM-level, orthogonal to the payload type. The voice just holds **Pattern Sound**.
- **Virtual machines (Balistes/Selene/Odonus)** — separate voice *kinds* that emit directly; they never touched the pattern layer and are untouched here.
- **Multi-output (MIDI/OSC/CV)** — fits *better*: typed fields read per binding.

Migration

Keep the verb signatures as **sugar over SoundPattern**, so most code migrates mechanically and much keeps compiling:

- **mini/drum** still take a String; **drum = s <<< mini.degree/note** still take a String/**Pattern.on vDrums kit (drum "...")** still type-checks.
- **every 8 rev (drum "...")** works (a **SoundPattern** is a **Pattern**).

- **PitchedPart/DrumPart** bodies become **SoundPattern** (or unify into one **Part**); **on**'s instances build them; the sidecar **params :: Map String (Pattern String)** **goes away** — folded into **Sound**.
- The dispatcher's per-event velocity (step 1, done: **gain**→velocity for MidiNote/MidiDrumKit) becomes "read **gain** from the typed **Sound**" instead of a string params map.
- The Calypso generator emits **SoundPattern** forms (**drum "..." # gain "..."**).

Breadth is the real cost: every session (Fugue, Full, ZR, Wired, ES-9 ones) + the generator + Studio bindings touch this. Mitigated by keeping the verbs as sugar.

Phasing

1. **Types + combinators (pure, compiles in isolation)**. **Sound, Pitch, Token**; the control verbs; **#** typed merge; parsers as sugar. No wiring yet.
2. **Voice payload → SoundPattern**. Rewrite **Tidal/Voice.purs computeDiscrete** to query **Pattern Sound** and extract per-event (source token, pitch, controls); resolve **Degree** at emit; drop the string param sidecar. Dispatch reads typed fields.
3. **Parts + bindings + sessions**. **on/PitchedPart/DrumPart** over **Sound**; bindings read typed fields; migrate existing sessions mechanically.
4. **Accents fall out + the original goal**. **gain** is a field; wire the Calypso lowering to emit **# gain** from **StepProfile** strength; play the Lumbeat / dembow patterns with real dynamics into Ableton (MIDI velocity). Verify.
5. **(Optional, later) sample-trigger binding** for Rample/Digitakt alongside Dirt.

Risks / open questions

1. **computeDiscrete rewrite** is the core surgery — bounded but central; the rig must stay playable, so do it behind a compile + a focused reload.
2. **# merge semantics** — define right-biased field override precisely; decide if/when to add typed **|+||*|**.
3. **IsString (Pattern a)** for mini-notation literals — ergonomic call, orthogonal to the type model.
4. **mini vs s/drum roles** — **mini "..."** yields a bare value pattern; is it only ever used *inside* a control verb, or also standalone? Clarify so the surface is coherent.
5. **Migration breadth** — the size, not a blocker; sugar limits the blast radius.
6. **Sample-addressing** across Dirt/Rample/Digitakt — the one place the sample abstraction may not fully unify; keep it a binding concern.

Relationship to prior work

- Step 1 (dispatcher **gain**→velocity) is **done** and folds in (becomes "read the typed **gain** field").
- **neutral-accent-plan.md** is **superseded**: accents are now **Sound.gain**.
- After this lands, return to the **Lumbeat AfroCuban patterns** — the 4-level velocity maps directly onto **gain**, and the per-step strength → **gain** story is exactly the StepProfile → accent path.